# CS {4/6}290 & ECE {4/6}100 - Spring 2021

# Homework 2: Caches

Dr. Thomas Conte

Due: February 12th, 2021 @ 11:59 PM

## Rules

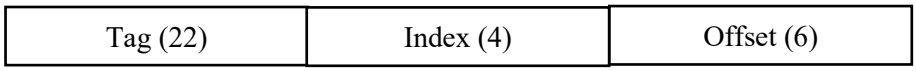
- **This is an individual assignment. ABSOLUTELY NO COLLABORATION IS PERMITTED**. All cases of honor code violations will be reported to the Dean of Students.
- The due date at the top of the assignment is final. Late assignments will not be accepted.
- Please use office hours for getting your questions answered. If you are unable to make it to office hours, please email the TAs.
- Make sure you **show your work and give explanations** for all questions unless otherwise stated.
- All answers must be **handwritten or typed**, and an electronic copy (PDF format) uploaded to Canvas before the due date.
- **Make sure your answers are written within the boxes we have provided. We will be using Gradescope for grading and answers outside the boxes will NOT be recognized by the system.**
- **There will be a point penalty for not following the above rule.**

# Question 0 – Cache Warmup

(a) Assume a cache with (C = 10, B = 6, S = 0). Given that this cache implements a write-allocate, write-back policy, and system addresses are 32-bits wide:

(i) How many bits make up the *Tag*, *Index*, and *Block Offset* of an address accessing this Cache? (i.e. show the address split for accessing this cache)

Assuming byte addressability for this question.
Index: C - B - S = 4 bits
Offset: 6 bits
Tag: 32 - offset - index = 22 bits




(ii) How many **bits** are there in the *Data Store* of the Cache?

Size of Data Store: 2^4 * 2^6 * 8 = 2^13 bits = 8192 bits

(iii) How many **bits** are there in the *Tag Store* of this Cache?

Assuming our cache has both valid and dirty bits.

Tag + valid + dirty: 24 bits
Number of tags: $2^4 = 16$
Size of Tag Store: 24 * 16 = 384 bits

(b) Consider the C, B and S parameters of a Cache. What happens to compulsory, capacity and conflict misses if only the following parameter is changed (the other two are kept the same)?

(i) C is increased

Compulsory miss: no change (since the total size of memory doesn't change)
Capacity miss: decrease
Conflict miss: decrease

(ii) S is increased

Compulsory miss: no change
Capacity miss: no change
Conflict miss: decrease

(iii) B is increased

Compulsory miss: decrease
Capacity miss: increase
Conflict miss: increase

# Question 1 – Hand to Hand Caches

Consider the memory address trace below for a 32-bit address size systeMiss:

```
ST 0x0ECE4100
LD 0x0ECE6100
LD 0x00C1CD88
LD 0x0054B0FA
LD 0x00ABCD89
ST 0x00CDEF9A
LD 0x000CB017
LD 0x1002ED12
LD 0x1054B09B
LD 0x00ABCD88
LD 0x00EF7300
ST 0x000CB680
```

Hand simulate the above cache accesses for the *(C,B,S)* cache configurations (a)-(c). Assume that caches use the LRU replacement policy (where applicable) and implement the write-back, write-allocate scheme. **Answer questions (i)-(iii) for each of the configurations:**

(i) Indicate which accesses would cause a miss. If an access is a miss, identify the type of miss.
(ii) How many write-backs happen for this stream? **Explain why?**
(iii) If the cache policy was *write-through instead of write-back*, how many **trips** to main memory will there be for this stream? **Explain why?**

(a) (9, 4, 0) tag(23) index(5) offset(4)
(b) (9, 4, 4) tag(27) index(1) offset(4)
**(c)** (9, 4, 2) tag(25) index(3) offset(4)

**Fill in this table to answer (i) {Hit : H, Miss : M} if (M) { Type}**

## We assume a cold start cache for this question.

| Stream | (a) | (b) | (c) |
|---|---|---|---|
| **ST 0x0ECE4100** | **Miss: compulsory** | **Miss: compulsory** | **Miss: compulsory** |
| LD 0x0ECE6100 | **Miss: compulsory** | **Miss: compulsory** | **Miss: compulsory** |
| LD 0x00C1CD88 | **Miss: compulsory** | **Miss: compulsory** | **Miss: compulsory** |
| LD 0x0054B0FA | **Miss: compulsory** | **Miss: compulsory** | **Miss: compulsory** |
| LD 0x00ABCD89 | **Miss: compulsory** | **Miss: compulsory** | **Miss: compulsory** |
| **ST 0x00CDEF9A** | **Miss: compulsory** | **Miss: compulsory** | **Miss: compulsory** |
| LD 0x000CB017 | **Miss: compulsory** | **Miss: compulsory** | **Miss: compulsory** |
| LD 0x1002ED12 | **Miss: compulsory** | **Miss: compulsory** | **Miss: compulsory** |
| LD 0x1054B09B | **Miss: compulsory** | **Miss: compulsory** | **Miss: compulsory** |
| LD 0x00ABCD88 | **Hit** | **Hit** | **Hit** |
| LD 0x00EF7300 | **Miss: compulsory** | **Miss: compulsory** | **Miss: compulsory** |
| **ST 0x000CB680** | **Miss: compulsory** | **Miss: compulsory** | **Miss: compulsory** |

## Space for Answer to Question 1 (ii) and (iii)

(ii) (a) How many write-backs happen for this stream?
1 write-back happened when `0x0ECE6100` kicked out `0x0ECE4100` because they both occupy set 16.

(ii) (b)
0 write-back happened. There is no cache blocked kicked out because 16 ways is enough to contain different tags.

(ii) (c)
1 write-back happened when `0x00EF7300` kicked out `0x0ECE4100` because of LRU policy when set 0 is full.

(iii) If the cache policy was *write-through instead of write-back*, how many trips to main memory will there be for this stream?

In case of a WTWA cache, each write miss would mean 2 trips to memory, one updating memory and the other bringing the block into cache.

3 * 2 (write misses) + 8 (read misses) = 14 trips to memory

# Question 2 – Cache Design

Given the memory trace from problem 1 with parameter B = 4:

(a) What is the smallest **direct-mapped** cache for which no misses besides *compulsory* occur?
(b) What is the smallest **Fully-Associative** cache for which no misses besides *compulsory* occur?
(c) What is the smallest **possible Set-Associative** Cache (Not fully associative) that completely avoids *Capacity* and *Conflict* misses? (Find the minimum C and S values to satisfy the constraint)

**Please show your work and explain**

## Space for Answer to Question 2

(a)
To eliminate conflict and capacity misses on a direct-mapped cache, we must make all the instructions' index completely different. Therefore, to accommodate `0x00C1CD88` and `0x00ABCD89`, whose address starts to differ at the 18th bit after the first four, we must have a index of length 14 bits for them to not occupy the same set.

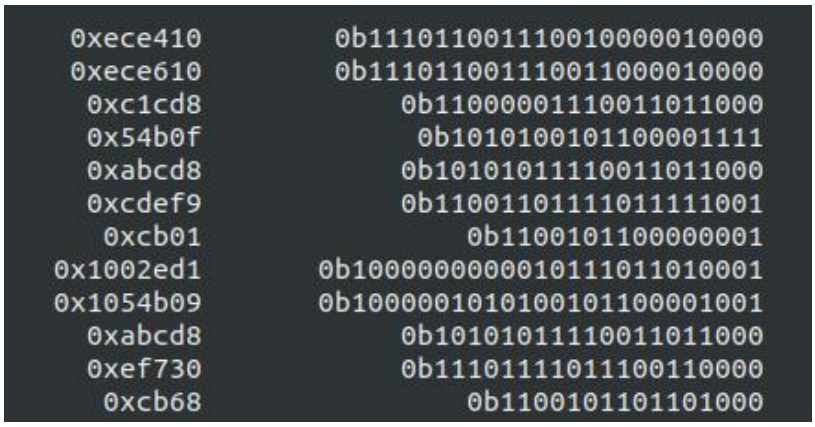
Cache size = 2^14 * 2^4 * 1 = 2^18 Bytes = 256KB. C = 18, B = 4, S = 0

(b)
For a fully-associative cache, we only need to make the number of blocks(ways) large enough to contain the different addresses. There are 11 unique addresses in total, so the fully-associative cache must be at least 16 blocks large.

Cache size = 16 * 2^4 = 2^8 = 256 Bytes. C = 8, B = 4, S = 4

(c)
Since there are 11 unique addresses only, we aim to make a cache that has only 16 blocks. A 2-set-associative and 8-way cache is the minimal I can think of. By observing the instruction stream with the least 4 bits truncated, we can observe that there are 6 instructions ending with '0'. With 1-bit index and 8 ways per set, we can put the 6 '0'-ending addresses into set 0, and the rest '1'-ending instructions into set 1. This will ensure no miss besides conflict misses.

Cache size = 8 * 2 * 2^4 = 2^8 = 256 Bytes  C = 8, S = 3, B = 4



# Question 3 – Cache Performance

A given memory hierarchy initially consists of an L1 cache and main memory. The L1 cache has a hit time of 2 cycles, and for a given workload a miss ratio (rate) of 15 percent. It takes 100 cycles to access main memory on average.

(a) What is the average access time for this system? **Show your work**

AAT = HT + MR * MP = 2 + 0.15 * 100 = 17 cycles

You are the chief memory architect for ACME computers and decide to improve your systems performance by considering the options listed in (b)-(d)

(b) Modification Option 1: Add a victim cache to the hierarchy. The victim cache has a miss ratio of 0.7. What is the average access time of the system now?

AAT = HT + MR * $MR_{VC}$ * MP = 2 + 0.15 * 0.7 * 100 = 12.5 cycles

(c) Modification Option 2: Add a second level cache (L2) with the following characteristics:
Hit Time = 8 Cycles, Miss Rate = 5 percent

AAT = HT1 + MR1 * MP1 = 2 + 0.15 * MP1
MP1 = HT2 + MR2 * MP2 = 8 + 0.05 * 100 = 13 cycles
AAT = 2 + 0.15 * 13 = 3.95 cycles

(d) Modification Option 3: You give up improving your system cache entirely and instead want to tell your main memory supplier to make faster memory… Yikes! How fast must main memory be such that when used with the original system the overall AAT is less than 4 cycles? *(It is okay to have a fractional value for memory latency for this problem)*

AAT = HT + MR * MP = 2 + 0.15 * MP < 4 cycles
MP < 40/3 cycles

The main memory's access time must be less than 40/3 cycles for the overall AAT to be less than 4 cycles.

(e) Which option do you choose after evaluating all of them or do you stick with your original memory system? **Please explain**

I choose to add a L2 cache. It provides significantly more AAT reduction compared to victim cache and the original design. Only faster memory can achieve similar improvement, but it requires 7.5x performance improvement on the main memory, which may not be feasible or cost-efficient.

Overall, adding a L2 cache is the best choice.